INTERNET AND WEB PROGRAMMING

LAB PROGRAMS

Name: Diksha Agrawal

Reg. Number: 23BEY10036

Slot: B11+B12+B13

Submitted To:

…….

Dr. Sreevani Maddukuri Ma'am

INDEX

S.no.	PROGRAMS	Pg. No.
1	PHP Basics-Array Operations	3
2	PHP Functions Implementation	6
3	Form Handling And Validation	9
4	File Handling System	12
5	Date And Time Application	14
6	MySQL Database Creation And Connectivity	16
7	Session Based Login System	21
8	Cookies Implementation	24
9	PHP Xml Processing	27
10	Ajax-Based Data Fetching System	30

Lab Programs

1. PHP Basics – Array Operations

Develop a PHP program to:

- Create indexed and associative arrays.
- Perform operations: insertion, deletion, sorting.
- Display elements using loops.

```
<?php

$categories = ["Electronics", "Furniture", "Apparel"];

$inventory = [

    "LP-101" => "Laptop",

    "CH-202" => "Office Chair",

    "TS-303" => "Cotton T-Shirt"

];


echo "### Initial State ###\n";

print_r($categories);

print_r($inventory);


echo "----------------------------------\n";


$categories[] = "Home Decor";


$inventory["MN-404"] = "Gaming Monitor";
```

```
unset($categories[1]);
$categories = array_values($categories);
unset($inventory["CH-202"]);
echo "### After Operations (Insertion, Deletion, Sorting) ###\n";

// 3. DISPLAYING ELEMENTS USING LOOPS

echo "\nCATEGORIES (Indexed Loop):\n";
for ($i = 0; $i < count($categories); $i++) {
    echo ($i + 1) . ". " . $categories[$i] . "\n";
}

echo "\nINVENTORY (Associative Loop):\n";
foreach ($inventory as $sku => $name) {
    echo "SKU: [$sku] | Product: $name\n";
}

?>
```

Output

Clear

```
### Initial State ###
Array
(
    [0] => Electronics
    [1] => Furniture
    [2] => Apparel
)
Array
(
    [LP-101] => Laptop
    [CH-202] => Office Chair
    [TS-303] => Cotton T-Shirt
)
----------------------------------
### After Operations (Insertion, Deletion, Sorting) ###

CATEGORIES (Indexed Loop):
1. Apparel
2. Electronics
3. Home Decor

INVENTORY (Associative Loop):
SKU: [TS-303] | Product: Cotton T-Shirt
SKU: [MN-404] | Product: Gaming Monitor
SKU: [LP-101] | Product: Laptop
```

2. PHP Functions Implementation

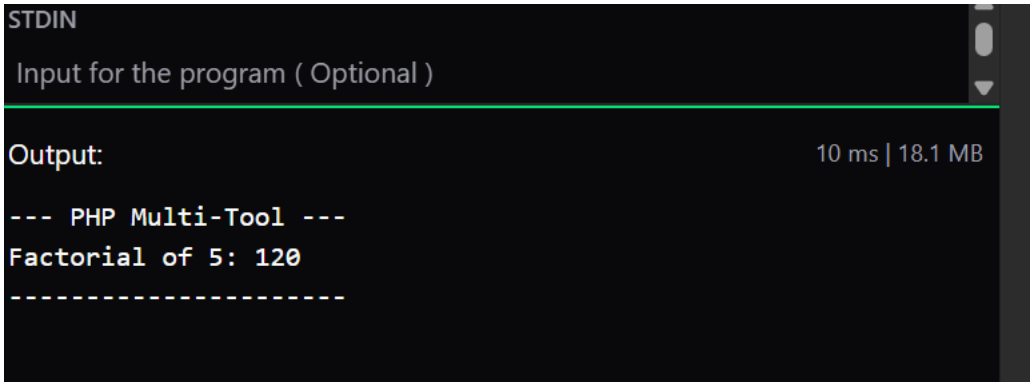
Write a PHP program to:

- Define user-defined functions for:
  - Factorial
  - Prime number check
  - String reversal
- Call functions based on user input.

```
<?php


$userChoice = 1;

$userInput  = 5;




function computeFactorial($n) {

    if ($n < 0) return "Undefined for negative numbers";


    $result = 1;

    for ($i = 1; $i <= $n; $i++) {

        $result *= $i;

    }

    return $result;

}


function isPrimeNumber($num) {

    if ($num < 2) return false;
```

```
    for ($i = 2; $i <= sqrt($num); $i++) {
        if ($num % $i == 0) return false;
    }
    return true;
}

function flipString($str) {
    $reversed = "";
    $length = strlen($str);
    for ($i = $length - 1; $i >= 0; $i--) {
        $reversed .= $str[$i];
    }
    return $reversed;
}

echo "--- PHP Multi-Tool ---\n";

switch ($userChoice) {
    case 1:
        echo "Factorial of $userInput: " . computeFactorial((int)$userInput);
        break;

    case 2:
        echo "Is $userInput Prime? " . (isPrimeNumber((int)$userInput) ? "Yes" : "No");
        break;
```

```
    case 3:
        echo "Original: $userInput\n";
        echo "Reversed: " . flipString((string)$userInput);
        break;

    default:
        echo "Invalid Option Selected.";
}

echo "\n----------------------\n";
?>
```



3. Form Handling and Validation

Create an HTML form and PHP script to:

- Accept Name, Email, Mobile number.
- Validate inputs (empty, format).
- Display sanitized user data.

```php
<?php
$userData = $errors = [];
$showData = false;

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $rawName  = trim($_POST['full_name'] ?? '');
    $rawEmail = trim($_POST['user_email'] ?? '');
    $rawPhone = trim($_POST['phone_no'] ?? '');


    // Name Validation
    if (empty($rawName)) {
        $errors[] = "Name is required.";
    } elseif (!preg_match("/^[a-zA-Z ]*$/", $rawName)) {
        $errors[] = "Only letters and spaces allowed in Name.";
    }

    // Email Validation
    if (empty($rawEmail)) {
        $errors[] = "Email is required.";
    } elseif (!filter_var($rawEmail, FILTER_VALIDATE_EMAIL)) {
        $errors[] = "Invalid email format.";
    }

    // Mobile Number Validation (Basic 10-digit check)
    if (empty($rawPhone)) {
        $errors[] = "Mobile number is required.";
    } elseif (!preg_match("/^[0-9]{10}$/", $rawPhone)) {
        $errors[] = "Mobile number must be exactly 10 digits.";
    }

    // 3. SANITIZATION (If no errors)
    if (empty($errors)) {
        $userData['name']  = htmlspecialchars($rawName);
        $userData['email'] = filter_var($rawEmail, FILTER_SANITIZE_EMAIL);
        $userData['phone'] = htmlspecialchars($rawPhone);
        $showData = true;
    }
}
?>

<!DOCTYPE html>
```

```
<html lang="en">
<head>
   <title>Secure Member Registration</title>
   <style>
      body { font-family: sans-serif; background: #f4f4f9; padding: 20px; }
      .container { max-width: 400px; background: white; padding: 20px; border-radius: 8px;
box-shadow: 0 2px 10px rgba(0,0,0,0.1); }
      .error { color: #d9534f; background: #f2dede; padding: 10px; border-radius: 4px;
margin-bottom: 15px; }
      .success { color: #3c763d; background: #dff0d8; padding: 15px; border-radius: 4px; }
      input { width: 100%; padding: 10px; margin: 8px 0; border: 1px solid #ccc; border-
radius: 4px; box-sizing: border-box; }
      button { background: #5cb85c; color: white; border: none; padding: 10px 15px; cursor:
pointer; width: 100%; }
   </style>
</head>
<body>

<div class="container">
   <h2>User Registration</h2>

   <?php if (!empty($errors)): ?>
      <div class="error">
         <?php foreach ($errors as $error) echo "• $error<br>"; ?>
      </div>
   <?php endif; ?>

   <?php if ($showData): ?>
      <div class="success">
         <h3>Data Received Successfully:</h3>
         <p><strong>Name:</strong> <?php echo $userData['name']; ?></p>
         <p><strong>Email:</strong> <?php echo $userData['email']; ?></p>
         <p><strong>Phone:</strong> <?php echo $userData['phone']; ?></p>
         <a href="">Submit another response</a>
      </div>
   <?php else: ?>
      <form method="post" action="<?php echo
htmlspecialchars($_SERVER["PHP_SELF"]);?>">
         <label>Full Name</label>
         <input type="text" name="full_name" placeholder="John Doe" value="<?php echo
$rawName ?? ''; ?>">

         <label>Email Address</label>
         <input type="text" name="user_email" placeholder="john@example.com"
value="<?php echo $rawEmail ?? ''; ?>">

         <label>Mobile Number</label>
         <input type="text" name="phone_no" placeholder="9876543210" value="<?php echo
$rawPhone ?? ''; ?>">
```

```
        <button type="submit">Register Now</button>
    </form>
  <?php endif; ?>
</div>

</body>
</html>
```

STDIN

Input for the program (Optional)

Output:

11 ms | 18.3 MB

```
<!DOCTYPE html>
<html lang="en">
<head>
    <title>Secure Member Registration</title>
    <style>
        body { font-family: sans-serif; background: #f4f4f9; padding: 20px; }
        .container { max-width: 400px; background: white; padding: 20px; border-radius: 8px; box-shadow: 0 2px 10px rgba(0,0,0,0.1); }
        .error { color: #d9534f; background: #f2dede; padding: 10px; border-radius: 4px; margin-bottom: 15px; }
        .success { color: #3c763d; background: #dff0d8; padding: 15px; border-radius: 4px; }
        input { width: 100%; padding: 10px; margin: 8px 0; border: 1px solid #ccc; border-radius: 4px; box-sizing: border-box; }
        button { background: #5cb85c; color: white; border: none; padding: 10px 15px; cursor: pointer; width: 100%; }
    </style>
</head>
<body>

<div class="container">
    <h2>User Registration</h2>


            <form method="post" action="HelloWorld.php">
            <label>Full Name</label>
```


```
</head>
<body>

<div class="container">
    <h2>User Registration</h2>


            <form method="post" action="HelloWorld.php">
            <label>Full Name</label>
```


```
            <input type="text" name="full_name" placeholder="John Doe" value="">

            <label>Email Address</label>
            <input type="text" name="user_email" placeholder="john@example.com" value="">

            <label>Mobile Number</label>
            <input type="text" name="phone_no" placeholder="9876543210" value="">

            <button type="submit">Register Now</button>
        </form>
    </div>

</body>
</html>
```

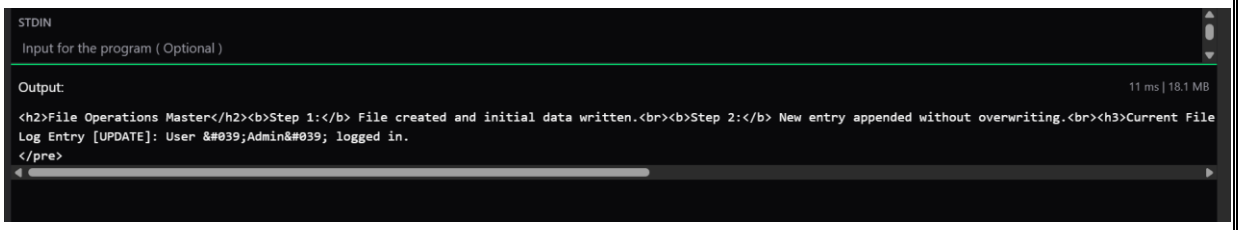
4. File Handling System

Develop a PHP program to:

- Create and write data into a file (`data.txt`).
- Read and display file contents.
- Append new data without overwriting.

```
<?php

$filename = "activity_log.txt";

echo "<h2>File Operations Master</h2>";

// 1. CREATE AND WRITE (OVERWRITE MODE)

$initialContent = "Log Entry [START]: System initialized at " . date('Y-m-d H:i:s') .
PHP_EOL;

$fileHandle = fopen($filename, "w");

if ($fileHandle) {

    fwrite($fileHandle, $initialContent);

    fclose($fileHandle);

    echo "<b>Step 1:</b> File created and initial data written.<br>";

}

// 2. APPEND NEW DATA (APPEND MODE)

$additionalData = "Log Entry [UPDATE]: User 'Admin' logged in." . PHP_EOL;

$fileHandle = fopen($filename, "a");
```

```
    if ($fileHandle) {
        fwrite($fileHandle, $additionalData);
        fclose($fileHandle);
        echo "<b>Step 2:</b> New entry appended without overwriting.<br>";
    }

    // 3. READ AND DISPLAY
    echo "<h3>Current File Contents:</h3>";
    if (file_exists($filename)) {
        $content = file_get_contents($filename);



        echo "<pre style='background: #eee; padding: 10px; border: 1px solid #ccc;'>";
        echo htmlspecialchars($content);
        echo "</pre>";
    } else {
        echo "Error: File could not be found.";
    }
?>
```



5. Date and Time Application

Write a PHP program to:

- Display current date and time.
- Calculate age from date of birth.
- Display day, month, year separately.

```
// 1. DISPLAY CURRENT DATE AND TIME
// Creating a DateTime object represents "now"
$currentMoment = new DateTime();

echo "<h2>System Chronology</h2>";
echo "<b>Current Server Time:</b> " . $currentMoment->format('Y-m-d H:i:s A') . "<br>";
echo "<b>Timezone:</b> " . $currentMoment->getTimezone()->getName() . "<br>";

echo "<hr>";

// 2. CALCULATE AGE FROM DATE OF BIRTH
$birthDateString = "1995-08-15";
$birthday = new DateTime($birthDateString);

// The 'diff' method calculates the exact gap between two DateTime objects
$ageDuration = $birthday->diff($currentMoment);

echo "<h3>Age Analysis</h3>";
echo "<b>Date of Birth:</b> " . $birthday->format('F j, Y') . "<br>";
echo "<b>Calculated Age:</b> " . $ageDuration->y . " Years, " . $ageDuration->m . "
Months, and " . $ageDuration->d . " Days.<br>";

echo "<hr>";

// 3. DISPLAY DAY, MONTH, YEAR SEPARATELY
// We extract specific components using format characters
echo "<h3>Component Breakdown</h3>";
$day   = $currentMoment->format('d'); // Day of the month (01 to 31)
$month = $currentMoment->format('F'); // Full textual representation of a month
$year  = $currentMoment->format('Y'); // A full numeric representation of a year (4 digits)

?>

<table border="1" cellpadding="10" style="border-collapse: collapse; text-align: center;">
   <tr style="background-color: #f2f2f2;">
      <th>Component</th>
      <th>Value</th>
      <th>Format Code</th>
   </tr>
   <tr>
```

```
        <td><strong>Day</strong></td>
        <td><?php echo $day; ?></td>
        <td><code>'d'</code></td>
    </tr>
    <tr>
        <td><strong>Month</strong></td>
        <td><?php echo $month; ?></td>
        <td><code>'F'</code></td>
    </tr>
    <tr>
        <td><strong>Year</strong></td>
        <td><?php echo $year; ?></td>
        <td><code>'Y'</code></td>
    </tr>
</table>
```

```
STDIN
Input for the program ( Optional )
<h2>System Chronology</h2><b>Current Server Time:</b> 2026-04-06 14:19:45 PM<br><b>Timezone:</b> UTC<br><hr><h3>Age Analysis</h3><b>Date of Birth:</b> August 15,
<table border="1" cellpadding="10" style="border-collapse: collapse; text-align: center;">
    <tr style="background-color: #f2f2f2;">
        <th>Component</th>
        <th>Value</th>
        <th>Format Code</th>
    </tr>
    <tr>
        <td><strong>Day</strong></td>
        <td>06</td>
        <td><code>'d'</code></td>
    </tr>
    <tr>
        <td><strong>Month</strong></td>
        <td>April</td>
        <td><code>'F'</code></td>
    </tr>
    <tr>
        <td><strong>Year</strong></td>
        <td>2026</td>
        <td><code>'Y'</code></td>
    </tr>
</table>
```

6. MySQL Database Creation and Connectivity

Develop a PHP program to:

- Create a MySQL database and table (`students`).
- Insert, update, delete records using PHP.
- Display records in tabular format.

```
<?php

$host    = 'localhost';

$user    = 'root';

$pass    = ''; // Update with your password

$db_name = 'UniversityDB';


try {

    $pdo = new PDO("mysql:host=$host", $user, $pass);

    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);


    $pdo->exec("CREATE DATABASE IF NOT EXISTS `$db_name` DEFAULT
CHARACTER SET utf8mb4");

    $pdo->exec("USE `$db_name` text");


    $tableSql = "CREATE TABLE IF NOT EXISTS students (

        id INT AUTO_INCREMENT PRIMARY KEY,

        full_name VARCHAR(100) NOT NULL,

        email VARCHAR(100) UNIQUE,

        grade DECIMAL(3,2),

        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

    )";
```

```
$pdo->exec($tableSql);


$action = $_GET['action'] ?? 'read';

$message = "";


switch ($action) {

    case 'create':

        $stmt = $pdo->prepare("INSERT INTO students (full_name, email, grade) VALUES
(?, ?, ?)");

        $stmt->execute(['John Doe', 'john@example.edu', 3.85]);

        $message = "Record added successfully!";

        break;


    case 'update':

        $stmt = $pdo->prepare("UPDATE students SET grade = ? WHERE id = ?");

        $stmt->execute([4.00, 1]); // Updates ID 1 to a 4.00 GPA

        $message = "Record updated!";

        break;


    case 'delete':

        $idToDelete = $_GET['id'] ?? 0;

        $stmt = $pdo->prepare("DELETE FROM students WHERE id = ?");

        $stmt->execute([$idToDelete]);

        $message = "Record #$idToDelete removed.";

        break;

}
```

```
} catch (PDOException $e) {
    die("Database Error: " . $e->getMessage());
}
?>

<!DOCTYPE html>
<html>
<head>
    <title>Advanced Student Portal</title>
    <style>
        body { font-family: sans-serif; margin: 40px; color: #333; }
        table { width: 100%; border-collapse: collapse; margin-top: 20px; }
        th, td { padding: 12px; border: 1px solid #ddd; text-align: left; }
        th { background: #2c3e50; color: white; }
        .alert { padding: 10px; background: #d4edda; border: 1px solid #c3e6cb; margin-
bottom: 20px; }
        .btn { padding: 8px 12px; text-decoration: none; border-radius: 4px; font-size: 13px; }
        .btn-add { background: #27ae60; color: white; }
        .btn-del { background: #e74c3c; color: white; }
    </style>
</head>
<body>

    <h1>Registry Management</h1>
```

```
<?php if($message): ?>
    <div class="alert"><?php echo $message; ?></div>
<?php endif; ?>


<div style="margin-bottom: 20px;">
    <a href="?action=create" class="btn btn-add">+ Insert Sample Student</a>
    <a href="index.php" class="btn" style="background:#3498db; color:white;">Refresh
List</a>
</div>


<table>
    <thead>
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Email Address</th>
            <th>GPA</th>
            <th>Actions</th>
        </tr>
    </thead>
    <tbody>
        <?php
        $stmt = $pdo->query("SELECT * FROM students ORDER BY id DESC");
        while ($row = $stmt->fetch(PDO::FETCH_ASSOC)):
        ?>
            <tr>
```

```
                <td>#<?php echo $row['id']; ?></td>
                <td><strong><?php echo htmlspecialchars($row['full_name']); ?></strong></td>
                <td><?php echo htmlspecialchars($row['email']); ?></td>
                <td><?php echo $row['grade']; ?></td>
                <td>
                  <a href="?action=delete&id=<?php echo $row['id']; ?>"
                     class="btn btn-del"
                     onclick="return confirm('Archive this record?')">Delete</a>
                </td>
              </tr>
            <?php endwhile; ?>
          </tbody>
        </table>

    </body>
    </html>
```

7. Session-Based Login System

Create a login system using PHP:

- Validate user credentials from database.
- Start session on login.
- Restrict access to dashboard page.
- Implement logout using session destroy.

(Aligned with lab-style applications like those listed on page 3 such as shopping cart using sessions.)

```
CREATE TABLE IF NOT EXISTS users (
   id INT AUTO_INCREMENT PRIMARY KEY,
   username VARCHAR(50) UNIQUE NOT NULL,
   password VARCHAR(255) NOT NULL
);

-- Insert a test user (Password is 'admin123')
INSERT INTO users (username, password) VALUES ('admin', 'admin123');
```

2. The Login Page (login.php)
This file handles both the form display and the credential verification.
PHP

```
<?php
session_start();


if (isset($_SESSION['user_id'])) {
   header("Location: dashboard.php");
   exit();
}

$error = "";

if ($_SERVER["REQUEST_METHOD"] == "POST") {
   $conn = new mysqli("localhost", "root", "", "AcademyDB");

   $user = $conn->real_escape_string($_POST['username']);
   $pass = $_POST['password'];

   $result = $conn->query("SELECT id, password FROM users WHERE username =
'$user'");

   if ($result->num_rows === 1) {
      $row = $result->fetch_assoc();
      if ($pass === $row['password']) {
         $_SESSION['user_id'] = $row['id'];
         $_SESSION['username'] = $user;
```

```
            header("Location: dashboard.php");
            exit();
        } else {
            $error = "Invalid password.";
        }
    } else {
        $error = "User not found.";
    }
    $conn->close();
}
?>

<!DOCTYPE html>
<html>
<head><title>Login System</title></head>
<body style="font-family: sans-serif; display: flex; justify-content: center; padding-top:
50px;">
    <div style="border: 1px solid #ccc; padding: 20px; border-radius: 8px;">
        <h2>Login</h2>
        <?php if($error) echo "<p style='color:red;'>$error</p>"; ?>
        <form method="POST">
            <input type="text" name="username" placeholder="Username" required><br><br>
            <input type="password" name="password" placeholder="Password"
required><br><br>
            <button type="submit">Login</button>
        </form>
    </div>
</body>
</html>
```

3. The Protected Dashboard (dashboard.php)

This page checks if the user_id session variable exists. If not, it kicks the user back to the login page.

PHP

```
<?php
session_start();

// RESTRICT ACCESS: If no session exists, redirect to login
if (!isset($_SESSION['user_id'])) {
    header("Location: login.php");
    exit();
}
?>

<!DOCTYPE html>
<html>
<head><title>Dashboard</title></head>
<body>
```

```
    <h1>Welcome, <?php echo htmlspecialchars($_SESSION['username']); ?>!</h1>
    <p>This is a restricted area only visible to logged-in users.</p>
    <hr>
    <a href="logout.php" style="color: red;">Click here to Logout</a>
</body>
</html>
```

4. The Logout Script (logout.php)

This script cleans up all session data and redirects the user.

PHP

```
<?php
session_start();

$_SESSION = array();

session_destroy();


header("Location: login.php?msg=logged_out");
exit();
?>
```

8. Cookies Implementation

Write a PHP program to:

- Store user preferences (e.g., username) in cookies.
- Retrieve and display cookie data.
- Set expiry time and delete cookies.

```
<?php


$cookie_name = "preferred_user";

$status_message = "";


// 1. SETTING THE COOKIE

if (isset($_POST['set_cookie'])) {

    $user_value = htmlspecialchars($_POST['username']);

    $expiry = time() + 86400; // Current time + 24 hours


    setcookie($cookie_name, $user_value, $expiry, "/");


    header("Refresh:0");

}


// 2. DELETING THE COOKIE

if (isset($_POST['delete_cookie'])) {

    setcookie($cookie_name, "", time() - 3600, "/");

    header("Refresh:0");

}
```

```
// 3. RETRIEVING THE COOKIE

$saved_name = isset($_COOKIE[$cookie_name]) ? $_COOKIE[$cookie_name] : "Guest";

?>


<!DOCTYPE html>

<html>

<head>

    <title>PHP Cookie Lab</title>

    <style>

        body { font-family: sans-serif; text-align: center; padding-top: 50px; }

        .box { display: inline-block; padding: 20px; border: 2px solid #333; border-radius: 10px;
background: #f9f9f9; }

        .welcome { color: #2c3e50; font-size: 1.5em; margin-bottom: 20px; }

    </style>

</head>

<body>


    <div class="box">

        <div class="welcome">

            Hello, <strong><?php echo $saved_name; ?></strong>!

        </div>


        <?php if ($saved_name === "Guest"): ?>

            <form method="POST">

                <input type="text" name="username" placeholder="Enter your name" required>

                <button type="submit" name="set_cookie">Remember Me</button>
```

```
        </form>

    <?php else: ?>

        <p>The browser remembers you via a cookie.</p>

        <form method="POST">

            <button type="submit" name="delete_cookie" style="color: red;">Forget
Me</button>

        </form>

    <?php endif; ?>

  </div>


</body>

</html>
```

STDIN

Input for the program (Optional)

```
<!DOCTYPE html>
<html>
<head>
    <title>PHP Cookie Lab</title>
    <style>
        body { font-family: sans-serif; text-align: center; padding-top: 50px; }
        .box { display: inline-block; padding: 20px; border: 2px solid #333; border-radius: 10px; background: #f9f9f9; }
        .welcome { color: #2c3e50; font-size: 1.5em; margin-bottom: 20px; }
    </style>
</head>
<body>

    <div class="box">
        <div class="welcome">
            Hello, <strong>Guest</strong>!
        </div>

                <form method="POST">
            <input type="text" name="username" placeholder="Enter your name" required>
            <button type="submit" name="set_cookie">Remember Me</button>
        </form>
        </div>

</body>
</html>
```

9. PHP with XML Processing

Develop a PHP program to:

- Create an XML file (`students.xml`).
- Add student details dynamically.
- Parse and display XML data using DOM or SimpleXML.

```
<?php


class StudentManager {

    private $filename;


    public function __construct($filename = 'students.xml') {

        $this->filename = $filename;

        if (!file_exists($this->filename)) {

            $this->initializeXml();

        }

    }


    private function initializeXml() {

        $xml = new SimpleXMLElement('<university></university>');

        $xml->asXML($this->filename);

    }


    public function addStudent($id, $name, $major, $gpa) {

        $xml = simplexml_load_file($this->filename);
```

```
    $student = $xml->addChild('student');
    $student->addAttribute('id', $id);
    $student->addChild('name', htmlspecialchars($name));
    $student->addChild('major', $major);
    $student->addChild('gpa', $gpa);

    $dom = new DOMDocument('1.0');
    $dom->preserveWhiteSpace = false;
    $dom->formatOutput = true;
    $dom->loadXML($xml->asXML());

    return $dom->save($this->filename);
}

public function displayStudents() {
    if (!file_exists($this->filename)) return;

    $xml = simplexml_load_file($this->filename);

    echo "<h2>Student Records</h2>";
    echo "<table border='1' cellpadding='10' style='border-collapse: collapse;'>
            <tr style='background-color: #f2f2f2;'>
                <th>ID</th>
                <th>Name</th>
```

```
                <th>Major</th>
                <th>GPA</th>
            </tr>";

        foreach ($xml->student as $s) {
            echo "<tr>
                <td>{$s['id']}</td>
                <td>{$s->name}</td>
                <td>{$s->major}</td>
                <td>{$s->gpa}</td>
            </tr>";
        }
        echo "</table>";
    }
}

$manager = new StudentManager('students.xml');
$manager->addStudent("S101", "Alice Hamilton", "Cyber Security", "3.9");
$manager->addStudent("S102", "Bob Vance", "Refrigeration", "3.2");
$manager->addStudent("S105", "Charlie Day", "Law", "2.8");
$manager->displayStudents();

?>
```

10. AJAX-Based Data Fetching System

Develop a web application to:

- Fetch student/product data from MySQL using PHP.
- Use AJAX to display results without page reload.
- Implement live search functionality.

1. The Database Setup

Ensure you have the AcademyDB and students table from our previous exercise. If not, run this SQL:

SQL

```
CREATE DATABASE IF NOT EXISTS AcademyDB;

USE AcademyDB;

CREATE TABLE IF NOT EXISTS students (

    id INT AUTO_INCREMENT PRIMARY KEY,

    fullname VARCHAR(100),

    major VARCHAR(100)

);

-- Add some sample data

INSERT INTO students (fullname, major) VALUES ('Alice Johnson', 'Computer Science'), ('Bob Smith', 'Mathematics'), ('Charlie Brown', 'Physics');
```

2. The Backend Processor (search_engine.php)

This script acts as an "API endpoint." It listens for a search query, talks to the database, and returns the results.

PHP

```
<?php

$conn = new mysqli("localhost", "root", "", "AcademyDB");
```

```
if ($conn->connect_error) {

    die("Connection failed: " . $conn->connect_error);

}

$query = isset($_GET['q']) ? $conn->real_error_string($_GET['q']) : '';


$sql = "SELECT * FROM students WHERE fullname LIKE '%$query%' OR major LIKE
'%$query%'";

$result = $conn->query($sql);


if ($result->num_rows > 0) {

    echo "<table border='1' width='100%'>

            <tr style='background:#f4f4f4'><th>ID</th><th>Name</th><th>Major</th></tr>";

    while($row = $result->fetch_assoc()) {

        echo "<tr>

                <td>{$row['id']}</td>

                <td>{$row['fullname']}</td>

                <td>{$row['major']}</td>

               </tr>";

    }

    echo "</table>";

} else {

    echo "<p style='color:red;'>No matching students found.</p>";

}


$conn->close();
```

```
?>
```

3. The Frontend Interface (index.php)

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <title>AJAX Live Student Search</title>

    <style>

        body { font-family: 'Segoe UI', sans-serif; padding: 40px; background: #eef2f3; }

        .search-box {

            padding: 15px; width: 100%; max-width: 500px;

            border: 2px solid #007bff; border-radius: 25px; outline: none;

            font-size: 16px; margin-bottom: 20px;

        }

        #results { background: white; padding: 20px; border-radius: 10px; box-shadow: 0 4px
8px rgba(0,0,0,0.1); }

        h2 { color: #333; }

    </style>

</head>

<body>



    <h2>Academic Directory Live Search</h2>

    <input type="text" id="searchInput" class="search-box" placeholder="Start typing a name
or major..." onkeyup="fetchData(this.value)">



    <div id="results">
```

```
    <p>Results will appear here as you type...</p>
</div>

<script>
    /**
     * AJAX Function using Fetch API
     */
    function fetchData(searchTerm) {
        // Target the results div
        const resultDiv = document.getElementById('results');

        // If input is empty, clear the results
        if (searchTerm.length === 0) {
            resultDiv.innerHTML = "<p>Results will appear here as you type...</p>";
            return;
        }

        // AJAX FETCH REQUEST
        fetch('search_engine.php?q=' + encodeURIComponent(searchTerm))
            .then(response => response.text()) // Convert response to text (HTML)
            .then(data => {
                // Update the div without reloading the whole page
                resultDiv.innerHTML = data;
            })
            .catch(error => {
```

```
                console.error('Error fetching data:', error);
            });
        }
    </script>


</body>
</html>
```

STDIN

Input for the program (Optional)

Output: 60 ms | 18.0 MB

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>AJAX Live Student Search</title>
    <style>
        body { font-family: 'Segoe UI', sans-serif; padding: 40px; background: #eef2f3; }
        .search-box {
            padding: 15px; width: 100%; max-width: 500px;
            border: 2px solid #007bff; border-radius: 25px; outline: none;
            font-size: 16px; margin-bottom: 20px;
        }
        #results { background: white; padding: 20px; border-radius: 10px; box-shadow: 0 4px 8px rgba(0,0,0,0.1); }
        h2 { color: #333; }
    </style>
</head>
<body>

    <h2>Academic Directory Live Search</h2>
    <input type="text" id="searchInput" class="search-box" placeholder="Start typing a name or major..." onkeyup="fetchData(this.value)">

    <div id="results">
        <p>Results will appear here as you type...</p>
    </div>
```

STDIN

Input for the program (Optional)

```
<script>
    /**
     * AJAX Function using Fetch API
     */
    function fetchData(searchTerm) {
        // Target the results div
        const resultDiv = document.getElementById('results');

        // If input is empty, clear the results
        if (searchTerm.length === 0) {
            resultDiv.innerHTML = "<p>Results will appear here as you type...</p>";
            return;
        }

        // AJAX FETCH REQUEST
        fetch('search_engine.php?q=' + encodeURIComponent(searchTerm))
            .then(response => response.text()) // Convert response to text (HTML)
            .then(data => {
                // Update the div without reloading the whole page
                resultDiv.innerHTML = data;
            })
            .catch(error => {
                console.error('Error fetching data:', error);
            });
    }
</script>
```